

```

#include <stdio.h>

#include <stdlib.h>

#include <omp.h>

// Recursive function to calculate n-th Fibonacci number using OpenMP tasks
int fib_parallel(int n) {
    if (n <= 1) return n;

    int x, y;

    // Spawn task for fib(n-1)
    #pragma omp task shared(x) firstprivate(n)
    x = fib_parallel(n - 1);

    // Spawn task for fib(n-2)
    #pragma omp task shared(y) firstprivate(n)
    y = fib_parallel(n - 2);

    // Wait for both sub-tasks to complete before summing
    #pragma omp taskwait

    return x + y;
}

// Function to print the entire Fibonacci sequence up to n using tasks
void calculate_fib_parallel(int n) {
    int fib_numbers[n + 1];

    printf("Parallel Fibonacci sequence (Tasks): ");

    #pragma omp parallel

```

```

{
    // One single thread creates tasks to avoid duplicate work
    #pragma omp single
    {
        for (int i = 0; i <= n; i++) {
            // Task directive to compute individual term
            #pragma omp task shared(fib_numbers) firstprivate(i)
            {
                fib_numbers[i] = fib_parallel(i);
            }
        }
        // Wait for all iteration tasks to finish
        #pragma omp taskwait
    }
}

// Print the computed numbers sequentially
for (int i = 0; i <= n; i++) {
    printf("%d ", fib_numbers[i]);
}
printf("\n");
}

```

// Sequential Fibonacci implementation for comparison

```

void fib_serial(int n) {
    int fib_numbers[n + 1];
    fib_numbers[0] = 0;
    if (n > 0) fib_numbers[1] = 1;

    printf("Serial Fibonacci sequence      : ");
    for (int i = 2; i <= n; i++) {

```

```

        fib_numbers[i] = fib_numbers[i - 1] + fib_numbers[i - 2];
    }

    for (int i = 0; i <= n; i++) {
        printf("%d ", fib_numbers[i]);
    }
    printf("\n");
}

int main() {
    int n;

    printf("Enter the number of Fibonacci terms (n): ");
    if (scanf("%d", &n) != 1 || n < 0) {
        printf("Invalid input! Please enter n >= 0.\n");
        return 1;
    }

    printf("\n-----\n");

    // ----- SERIAL EXECUTION -----
    double start_serial = omp_get_wtime();
    fib_serial(n);
    double end_serial = omp_get_wtime();
    double time_serial = end_serial - start_serial;

    // ----- PARALLEL EXECUTION -----
    double start_parallel = omp_get_wtime();
    calculate_fib_parallel(n);
    double end_parallel = omp_get_wtime();
    double time_parallel = end_parallel - start_parallel;

```

```
// ----- TIMING COMPARISON -----  
printf("-----\n");  
printf("Serial Execution Time : %.6f seconds\n", time_serial);  
printf("Parallel Execution Time : %.6f seconds\n", time_parallel);  
printf("-----\n");  
  
return 0;  
}
```